Design layer

the whole layer, as one document

Contents

1 JSON contracts 4

1.1 Foundation 4
1.2 Pending updates 6
1.3 System at a glance 7
1.4 Whole model 8
1.5 Wire values 11
1.5.1 Exact numbers 11
1.5.2 Strict document admission 12
1.5.3 Rendering and standard JSON bridges 13
1.6 Typed codecs 13
1.6.1 Constructor acceptance 13
1.6.2 Record construction and dependent fields 14
1.6.3 Tagged union construction 15
1.6.4 Mappings and custom conversion 15
1.6.5 Paths and safe diagnostics 15
1.7 Runtime contracts 16
1.7.1 Schema discovery and public projection 16
1.7.2 Document loading profile 16
1.7.3 Normalization and schema matching 17
1.7.4 Validation and native conversion 18
1.8 Legacy compatibility 18
1.9 Source generation 19
1.9.1 Typed-definition generator 19
1.9.2 Schema materialization 20
1.9.3 Schema-driven generation obligations 20
1.10 Schema extension families 20
1.11 Verification and extension boundaries 21
1.11.1 Oracle and behavioral evidence 21
1.11.2 Native and generated boundaries 21
1.11.3 Documentation and build contracts 22
1.12 End-to-end walkthrough 22

Glossary 24

JSON contracts

1 JSON contracts

1.1 Foundation

One definition supplies typed conversion and an inspectable JSON contract.

GOAL

Describe schema-aware data once

Applications retain one Codec for JSON conversion, schema publication, and schema-aware composition. Runtime schema documents produce contracts for wire validation.

GOAL

Reject hostile input within explicit bounds

Strict text admission retains exact numbers and duplicate-key evidence. Limits bound input bytes, nesting, value count, and numeric tokens.

GOAL

Retain published data and broader capability scope

The frozen 1.x API remains available for its established formats and recursive types. Broader schemas and development-time native source generation remain intended capabilities with explicit unresolved contracts.

NO-GOAL

Execute or authorize application operations

Tool registration, provider admission, transport, job execution, workflow execution, and application policy belong to consumers. Ordinary JSON serialization can use standard Gleam JSON without this package.

NO-GOAL

Claim general JSON Schema conformance

The Finite profile has bounded independent evidence. A Draft 2020-12 dialect label does not imply support for every keyword or an arbitrary remote schema.

INVARIANT

Number comparison preserves mathematical value

Exact number comparison and integrality never pass through Float. Native construction and projection check target representation limits.

mechanism

INVARIANT

A witness retains the value that was validated

A Validated value retains an immutable value and the accepting normalized schema. A replacement value requires another validation.

mechanism

INVARIANT

Unknown schema never becomes inferred schema

A custom codec without a schema reports UnknownSchema. Every enclosing schema query propagates that absence.

mechanism

INVARIANT

Record decoding rejects extra and repeated fields

Closed record decoding rejects unknown members and repeated members. Strict text admission rejects repeated keys before native decoding.

mechanism

PRINCIPLE

Keep wire acceptance separate from native projection

A schema may accept an exact value that a requested native type cannot represent. Callers distinguish schema rejection, parse rejection, and native conversion failure.

PRINCIPLE

Keep callback ownership explicit

Caller mappings own their laws and application checks. A retained schema does not certify a callback's inverse, purity, or encoding behavior.

- Ownership and finite support are recorded in ADR-0001. Number policy is recorded in ADR-0002.

1.2 Pending updates

build **Broader schema families need concrete contracts** –

String constraints, broader numeric and collection constraints, arbitrary alternatives, numeric enums, aliases, and recursive references remain intended. Their concrete limits, annotation policies, and validation rules are unresolved; see Schema extension families.

ADR-0001

build **Runtime-schema native generation remains incomplete** –

Retain schema-driven native type and codec generation, stable selected names, shared definitions, nested and recursive forms, and CLI workflow. The dev-only typed-definition generator and schema materializer implement narrower contracts; see Source generation.

ADR-0008

ruling **Define encoding laws for contract.value_codec** –

Decoding validates; encoding returns the value unchanged, including values outside the schema. Do not extend the native codec-law claim to this adapter without deciding its encoding contract.

ADR-0006

verify **Retain complete compatibility verification** –

Published source, legacy wire, recursive references, error paths, numeric boundaries, and deliberate acceptance corrections need separate fixtures. Existing examples and frozen comparators do not prove complete 1.x compatibility.

ADR-0007

verify **Keep numerical oracle scope separate from projection** –

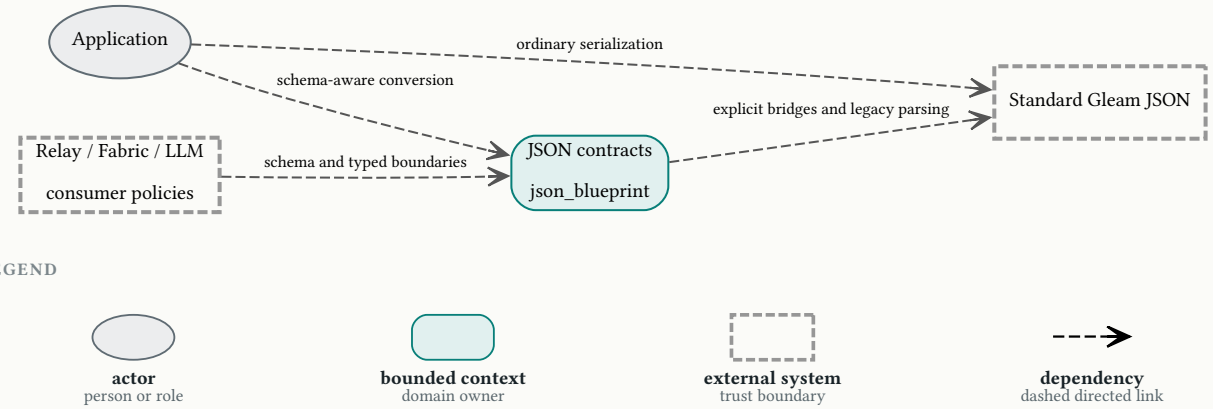
The finite schema manifest compares Boolean acceptance for its selected cases. Broader numerical evidence must compare exact schema acceptance separately from native overflow, safe-integer refusal, and intentional Float rounding.

ADR-0002

1.3 System at a glance

One context owns values, schemas, typed codecs, contracts, legacy compatibility, and generation definitions.

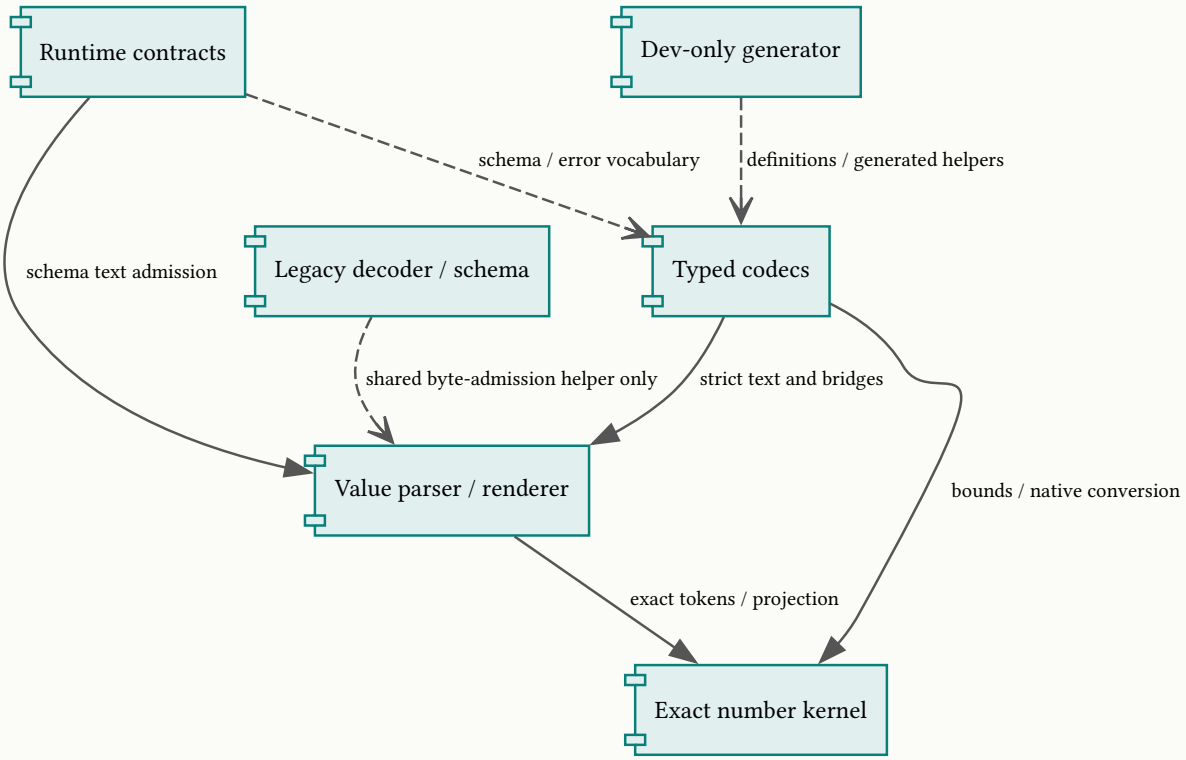
VIEWPOINT CONTEXT-OWNERSHIP · ALTITUDE L1 · BOUNDARY Application-owned composition



LEGEND

Arrows declare dependency direction. Consumers select their own protocol and provider admission policy.

- The package exports pure values and synchronous conversion operations. It starts no process, owns no connection, and stores no mutable runtime state.
- Local units are Wire values, Typed codecs, Runtime contracts, Legacy compatibility, Source generation, and Verification and extension boundaries. Each unit's failures and interactions are stated below.
- Relay owns protocol declarations and revision-specific generation. Fabric owns tool and model execution. LLM adapters own provider lowering and admission. These consumers inspect opaque schema children; they never acquire authority from a dialect label alone.

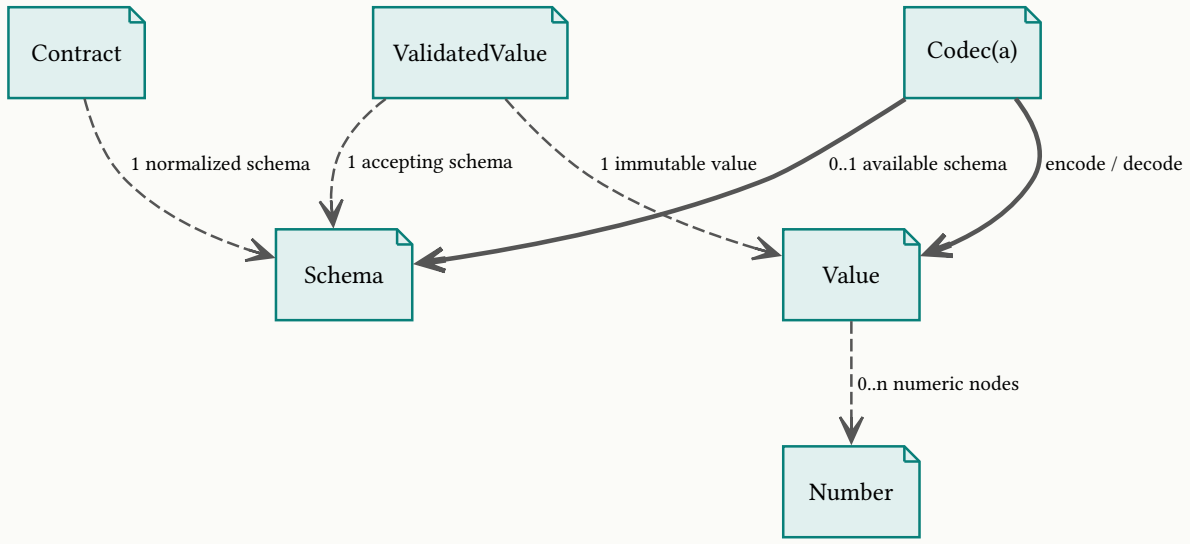


Generation is a build-time dependency. Application runtime needs the codec package and generated source, not the generator.

- No operation supplies authorization, identifiers for business entities, correlation, idempotency keys, retries, cancellation, or a wall-clock deadline. Applications own those capabilities where their operation requires them.
- Work is bounded only at the declared admission surfaces. A manually constructed Value, a caller callback, schema discovery, rendering, validation, and generation have no additional timer or allocation quota.

1.4 Whole model

All package-owned domain values are immutable. Schema identity is structural; native types remain caller-owned.



LEGEND



The diagram shows ownership and data relationships. Validation creates another immutable value; it does not mutate the input or register a global identity.

Number

value-object · immutable · JSON contracts · owned by Wire values

Target-independent mathematical numeric value.

ATTRIBUTES

derived **Normalized value** · SmallInteger | Decimal
Small integers have at most 15 decimal digits. Other values carry sign, nonzero coefficient digits without leading or trailing zeros, and a decimal exponent; zero has one positive representation.

RELATIONSHIPS

`1 : 0..n` Number may appear in Values and exact numeric schema bounds.

Value

value-object · immutable · JSON contracts · owned by Wire values

Ordered JSON tree that can retain duplicate-key evidence.

ATTRIBUTES

derived **Kind and content** · Null | Bool | String | Number | Array | Object
Parsing derives the content from admitted text. Public constructors also allow callers to supply content; those constructed values have no parse-admission proof.

RELATIONSHIPS

`1 : 0..n` Arrays own ordered Values. Objects own ordered name/Value pairs; the raw Object constructor permits repeated names.

Schema

value-object · immutable · JSON contracts · owned by Typed codecs

Checked finite structure with optional descriptions.

ATTRIBUTES

derived **Checked structure** · Opaque Tree
Codec discovery or document loading derives structure after checking definition invariants. The public projection never exposes a constructor for unchecked structure.

RELATIONSHIPS

1 : 0..n Property and variant records retain opaque child Schemas with exact name/requiredness and tag/payload associations.

Codec(a)

value-object · immutable · JSON contracts · owned by Typed codecs

Compiled native conversion and discoverable schema capability.

ATTRIBUTES

authored **Native directions** · a -> Result(Value, EncodeError); Value -> Result(a, DecodeError)
Constructors compose directions; custom callbacks are supplied by the caller.

derived **Definition and placeholder** · Deferred schema result; fn() -> a
Definition discovery is computed on demand. This is not a promised cached or constant-time schema lookup.

RELATIONSHIPS

1 : 0..1 A codec supplies one Schema when known. Its type parameter a is supplied by the native program, never by runtime schema text.

Contract

value-object · immutable · JSON contracts · owned by Runtime contracts

Normalized schema retained independently of native callbacks.

ATTRIBUTES

derived **Normalized acceptance** · Finite Tree
Normalization sorts properties, enum labels, and variants without changing associations. Equality for matching strips descriptions.

RELATIONSHIPS

1 : 0..n Validation may produce multiple witnesses. A contract retains no callback, process identity, or native type.

ValidatedValue

value-object · immutable · JSON contracts · owned by Runtime contracts

Immutable acceptance evidence for one retained value.

ATTRIBUTES

derived **Value and accepting schema** · Value × normalized Tree
Only successful validation constructs the public witness. Matching a codec checks normalized schema structure before its native decoder runs.

RELATIONSHIPS

`1 : 1` The witness retains exactly one value and one accepting schema; it cannot bless a modified value.

- Value objects have no persistent identifiers, mutable state fields, commands that change them, or emitted domain events. There is no vacant or revoked state to model: replacement discards the previous value and requires fresh evidence.
- Sum types model Value, SchemaView, typed failures, and union construction. Products model errors with paths, properties, variants, parse locations, and generated source results. Numeric bounds and enum membership are refinements on accepted values.
- Limits are immutable caller-authored settings. Their setters retain supplied values; nonpositive limits reject inputs according to the relevant bound rather than admitting an unbounded sentinel.

1.5 Wire values

This unit owns exact numbers, strict document admission, rendering, and explicit standard-JSON conversion.

Wire value boundary

responsibility

Preserve exact JSON meaning while admitting complete documents within explicit resource limits.

interface

number construction/projection; value.object, parse, parse_bits, to_string, to_json, decoder.

interactions

Codecs and schema loaders use strict admission. Standard-JSON bridges accept another parser's precision and duplicate-key policy.

invariants

No NaN or infinity is a Number. Checked object construction rejects repeated keys without converting the object to a map.

failure

ParseError retains location and reason; numeric construction and projection have separate typed outcomes.

1.5.1 Exact numbers

- Number parsing accepts the JSON numeric grammar: optional minus, integer digits with no leading zero except zero itself, optional nonempty fraction, and optional signed nonempty exponent. Whitespace and a leading plus are invalid.
- Normalization removes coefficient zeros and adjusts the decimal exponent without expanding the value. Comparison first distinguishes signs, then decimal magnitude and significant digits. Mathematical integrality follows normalized coefficient/exponent, so 1.0e2 is an integer.

Operation	Meaning	Failure or target rule
parse(token, limits)	Exact normalized decimal	InvalidSyntax, TokenTooLong, TooManySignificandDigits, ExponentOutOfRange
from_int / to_int	Checked exact native integer conversion	Erlang bignums; JavaScript safe integers only; projection has an output-digit bound
from_float / to_float	Shortest roundtrip decimal / nearest binary64	Nonfinite input or overflow rejects; tiny nonzero numbers may round to zero
from_float_exact / to_float_exact	Exact binary rational / exact mathematical binary64 equality	Overflow, underflow, inexactness, or invalid candidate remain distinct

- Default Number limits are 1,024 token bytes, 800 significant digits as written, and normalized exponent magnitude 1,200. They admit finite binary64 decimal expansions, including subnormals. Token and digit limits below one reject every token; a negative exponent bound rejects nonzero values.
- Native Int construction cannot recover precision already lost before the JavaScript boundary. The unsafe value is rejected even when a host has rounded it to another integer.

BEHAVIOR

Exact decimal survives comparison

Given The numbers 1.2300e2 and 123 were admitted within number limits.

When The application compares them.

Then The result is equality without binary rounding.

level: boundary area: Exact numbers

- The decimal 0.1 can project through the rounded conversion. Exact projection rejects it as FloatInexact; 0.5 projects exactly. Sources: number implementation and target and numeric cases.

1.5.2 Strict document admission

- Text parsing checks UTF-8 byte size before allocating the document tree. Byte parsing additionally reports the first invalid UTF-8 byte. A valid document has exactly one root value followed only by JSON whitespace; trailing content rejects.
- Each scalar, array, and object counts once, including the root and object member values. Object names do not add another value count. Depth counts nested arrays and objects; depth zero permits scalar roots only.

Bound	Default	Change
Input bytes	1,048,576	with_max_bytes
Container depth	64	with_max_depth
Values	262,144	with_max_elements
Number token bounds	1,024 / 800 / 1,200	with_number_limits

- Raising the byte limit does not raise the value limit. There is no streaming admission or request-wide time budget. The caller has already allocated input text; byte admission is a bound on further parsing, not on prior transport allocation.
- String admission validates escapes, four-digit Unicode escapes, and surrogate pairing. Literal control characters, malformed escapes, and unterminated strings reject. Object duplicate detection compares decoded names, so escaped equivalent names collide.
- A per-object dictionary tracks names before their values are admitted. Duplicate detection cannot be replaced by last-write-wins map construction. Arrays and objects retain input order, while parsed strings are copied so small retained slices do not hold the whole source binary.
- ParseError contains a zero-based byte offset and one-based line/column. Lines recognize LF, CR, and CRLF; columns count grapheme clusters. Parse errors retain no input text or unknown key string.

BEHAVIOR

The parser refuses repeated names

Given A document repeats the same decoded member name in one object.

When The application parses the document.

Then Parsing fails at the repeated name with DuplicateObjectKey.

Then No parsed tree is returned.

level: boundary area: Strict admission

BEHAVIOR

Value count is independently bounded

Given The next value would exceed the configured value count.

When The application parses a document within its byte bound.

Then Admission returns ElementLimitExceeded at the next value.

level: boundary area: Strict admission

- ADR-0003 records the independent admission bounds. Source: value parser. bounded parsing and grammar cases cover the admission boundary. Resource measurements remain operational guidance in README; they are not hard heap quotas.

1.5.3 Rendering and standard JSON bridges

- to_string emits compact JSON, escapes strings, and preserves ordered members. It does not normalize object order or reject duplicate members supplied through raw Object construction. Rendering has no independent byte bound.
- value.to_json first tries an exact native integer within 400 output digits, then a finite Float whose shortest roundtrip decimal compares equal to the Number. It returns the offending Number if no exact rendered standard-JSON form exists. This bridge criterion differs from exact binary64 equality: decimal 0.1 has an exact printed JSON form despite an inexact binary representation.
- codec.to_json retains the offending numeric node's Field/Index path as UnrepresentableNumber. Applications propagate that Result before entering APIs that require an infallible JSON encoder; replacing failure with null changes application data.
- value.decoder and codec.decoder consume a native parser's output. That parser owns size, duplicate names, member order, and already-rounded numeric precision. These bridges cannot restore discarded evidence or enforce strict text limits retroactively.
- Sources: rendering and bridge implementation and bridge cases.

1.6 Typed codecs

This unit composes bidirectional conversion for caller-owned native types and discovers the supported schema.

Codec boundary

responsibility

Encode and decode native a using one retained definition and known wire shape.

interface

Primitive codecs, collection codecs, record and union builders, mapping, custom, encode/decode, schema, check, placeholder.

interactions

Strict text parsing produces Value first. Schema publication exposes an opaque schema for consumer admission.

invariants

Built-in constructors preserve their declared wire shape. Unknown child schema propagates through every enclosing schema query.

failure

Value failures return path and Reason. Source definition mistakes panic at first affected use; check returns DefinitionError instead.

1.6.1 Constructor acceptance

Constructor	Native and JSON shape	Acceptance and encoding rule
string / bool	String / Bool	Exact JSON type; placeholder empty string / false
int	Native Int / mathematical integer	Decode output at most 24 digits and target-safe; encode checks native safety
float	Float / number	Decode nearest binary64, refuse overflow; encode shortest roundtrip decimal
number	Number / number	Exact passthrough without native projection
integer_between	Int / bounded mathematical integer	Inclusive safe-native bounds; projection digits derive from bounds
number_between	Number / bounded number	Inclusive exact comparisons
string_enum	Caller value / bare string label	Nonempty unique labels and unique native values; exact case-sensitive labels
pair	`#(a, b)` / two-element array	Exact length; failures retain index 0 or 1
list	List(a) / array	Decode and encode each item with its zero-based index
nullable	Option(a) / null or inner	None is null; Some cannot encode through inner as null
value	Value / any	Identity conversion with AnySchema; text admission still applies

Constructor	Native and JSON shape	Acceptance and encoding rule
success	r / empty object	Decode the retained r from a closed empty object; encoder ignores input

- A codec's accepted native domain can be narrower than its schema. Native Int digit limits and Float overflow are projections, not mathematical schema constraints. try_map may impose application checks that its preserved schema cannot express.
- For caller-authored mappings, a successful encode/decode roundtrip is a caller law. The type system proves neither inverse mappings nor equality to a schema language. A constant success codec has the retained constant as its valid native domain.
- DefinitionError distinguishes duplicate names/tags, empty union/enum, duplicate enum native values, reversed bounds, unsafe JavaScript integer bounds, and a field continuation that does not finish a record. Diagnostics name definition data rather than untrusted input.

BEHAVIOR

A nullable inner null cannot erase Some

Given A native Some value has an inner encoder that produces null.

When The application encodes it through nullable.

Then Encoding returns NullInsideNullable instead of losing its native distinction.

level: boundary area: Constructor acceptance

- Sources: constructors, codec cases, and public failure handling.

1.6.2 Record construction and dependent fields

- The Record builder binds each required field's value or each optional field's Option. success ends the continuation with the caller's record value. Getters are passed last with get:, so the continuation fixes the native record type before record access is checked.
- Encoding reads each getter in declaration order, encodes that field, then builds its continuation using the actual native field value. Decoding finds and decodes each named member, then builds the continuation using the decoded value. This permits a later field's application check to depend on earlier fields.
- Schema discovery follows continuations using each codec's Placeholder and None for optional fields. Names and schema constraints must be independent of those placeholder values; a dynamic application check may differ, but the declared JSON shape cannot.
- The source computes continuation properties on demand. Neither schema discovery nor arbitrary-width record processing promises constant time or linear time for all public definitions. Caller callbacks must remain deterministic and cheap where they are used to describe a schema.
- Optional fields omit None and decode absence as None. Required fields reject absence. Explicit null is rejected by a nonnullable inner codec; nesting optional_field with nullable yields None, Some(None), and Some(Some(x)) for absence, null, and a value.
- Record decoding visits declared fields before it checks unused members. A missing or invalid declared field can therefore precede an extra/repeated member failure. Runtime contract validation checks all member names before property values; the two first-error orders need not coincide.

BEHAVIOR

An optional field preserves present null

Given An optional field contains a nullable string codec.

When The application decodes an explicitly null field.

Then The result for that field is Some(None).

Then Re-encoding includes the field with null.

level: boundary area: Record construction

- A total mapped record has no record-builder continuation capability. A field continuation must remain a field/success record codec; terminating it in another shape is NotARecord.

- An Order total that differs from earlier decoded item quantities can return Custom at Field("total") when the check belongs inside that field's codec. Mapping the whole record reports at the record root. ADR-0004 records the builder and definition-error policy.

1.6.3 Tagged union construction

- union joins variant and unit_variant continuations with match. Each payload variant supplies a typed encoder to the final exhaustive native case; unit variants supply a retained tagged value. Duplicate tags and an empty union are definition errors.
- The Tagged union envelope has tag and value for payload variants, and tag alone for unit variants. All other fields reject. A payload variant requires value; a unit variant rejects even an explicitly null value.
- Decoding checks repeated and unknown member names before selecting tag. Missing tag, nonstring tag, unknown tag, missing payload, and invalid payload return distinct reasons at tag or value paths. Nested unions retain nested envelopes rather than implicitly flattening variants.

BEHAVIOR

A unit variant rejects a payload

Given The selected variant declares no payload.

When The application decodes an object containing value.

Then Decoding returns UnknownField at Field("value").

level: boundary area: Tagged unions

- Encoding trusts match to return the encoder for the selected native case. Reusing a Tagged value from another union is not dynamically certified against a registry; the author owns the union mapping laws. Tests exercise N-ary variants and unit shape in public API cases.

1.6.4 Mappings and custom conversion

- map takes total functions in both directions and derives its placeholder from the inner codec. try_map takes fallible functions plus an explicit placeholder; a conversion failure becomes Custom at the current codec path. Both preserve the inner schema without claiming that business checks are schema-expressible.
- custom accepts functions over Value, an Option(Schema), and a placeholder. None reports UnknownSchema; Some retains a checked schema but does not independently validate every callback result. Captured state and side effects remain callback-owner responsibility.
- placeholder returns description data for composing wrappers. It must never be encoded as fallback application data. A callback invoked during description may encounter this value even when application decoding never accepts it.

BEHAVIOR

Unknown custom schema propagates

Given A custom child codec supplies no schema.

When The application requests its enclosing record's schema.

Then The result is UnknownSchema.

Then The package does not infer an accepting document from callbacks.

level: boundary area: Custom conversion

1.6.5 Paths and safe diagnostics

- EncodeError and DecodeError each carry a root-to-leaf List(PathSegment) and structured Reason. Field(name) and Index(index) locate data. Empty paths identify the root; parse failures become InvalidJson at the root with their own byte location.
- Built-in rendered value reasons omit input values and unknown names from diagnostics. The typed path can still contain a name from the input. Custom messages render verbatim; the caller owns redaction and user-facing safety.
- Reason and ParseReason may gain minor-release alternatives. Callers use a fallback branch where exhaustiveness across upgrades matters. Schema errors, document errors, definition errors, parse errors, value errors, and number projections remain separate outcome families.

1.7 Runtime contracts

This unit loads the finite schema profile, validates immutable values, and checks structural agreement before native decoding.

Runtime contract boundary

responsibility

Interpret an explicit supported schema and preserve acceptance evidence.

interface

from_codec, total from_schema, parse, load, schema, same_schema, validate, value, decode, value_codec.

interactions

The strict parser owns text limits. The schema model owns checked structure. Native decoding retains the caller's codec.

invariants

Unsupported constraints never disappear. Matching preserves requiredness, tuple order, bounds, and variant associations.

failure

LoadError separates JSON admission from document rejection. ValidationError carries first failure path and Reason. Native mismatch is ContractMismatch.

1.7.1 Schema discovery and public projection

- codec.schema returns an opaque `Schema` or UnknownSchema after definition discovery. schema_value renders the schema node; schema_document adds the Draft 2020-12 declaration; schema_json renders the complete document as text.
- `Schema view` has fixed alternatives throughout 2.x: StringSchema, StringEnumSchema, IntSchema, IntegerRangeSchema, NumberSchema, NumberRangeSchema, BoolSchema, PairSchema, ListSchema, NullableSchema, ObjectSchema, UnionSchema, AnySchema, and OtherSchema.
- Descriptions are optional annotations read by codec.description; view looks through them. The latest description at one node replaces the earlier one. Child descriptions remain local to their nodes.
- A future minor-release kind reaches OtherSchema with its rendered schema object. A dedicated projection alternative requires a major release. PropertySchema and VariantSchema may gain fields; consumers read them by label. No current finite kind produces OtherSchema. `ADR-0005` records the evolution contract.

1.7.2 Document loading profile

- load requires an object root with exactly the supported Draft 2020-12 URI in `$schema`. It removes that declaration, interprets the remaining supported shape, checks definition invariants, then normalizes. Nested `$schema` declarations reject as NestedDialect.

Shape	Accepted members	Restriction
Any	Empty object, optionally description; nested true	Root still requires an object and `$schema`; false is not accepted
String / boolean	type; strings optionally enum	Enum is nonempty unique string labels
Integer / number	type; optional minimum and maximum	Both bounds or neither; inclusive and ordered; integer bounds project to safe native Int
Closed object	type, properties, required, additionalProperties	All three structure members explicit; additionalProperties must be false
Homogeneous array	type, items	Exactly one item schema
Pair	type, prefixItems, minItems, maxItems	Two prefix schemas and both lengths equal two; no items member
Nullable	anyOf	Exactly one plain null schema plus one supported object schema
Tagged union	type object, oneOf	Nonempty closed variants; tag const string; exactly declared payload presence

- description is permitted as an annotation on interpreted object schemas. Other keywords reject at their member path. The loader does not accept equivalent arbitrary forms, references, open objects, boolean false, standalone null, type arrays, or general anyOf/oneOf.
- Duplicate raw document members reject before interpretation. MalformedDocument reports observed keyword type and shape violations. UnsupportedDocument reports unhandled keywords, omitted profile-required members, and restrictions on otherwise valid schema forms. UnsupportedDialect retains the declared dialect; InvalidDefinition locates contradictions in the codec model such as reversed bounds or duplicate enum labels/tags.
- Object profile restrictions return UnsupportedDocument: omitted additionalProperties reports OpenObject; omitted properties or required and undeclared required names report UnsupportedSchemaForm at the member or name path. Schema-valued additionalProperties also rejects as UnsupportedSchemaForm. Explicit wrong member types and duplicate required names remain malformed. No omitted member is filled in to admit a new shape.
- Document errors describe the first profile rejection, not complete Draft 2020-12 meta-schema validation. UnsupportedDocument does not certify that the rest of the schema is valid. Boolean roots, missing dialect/type, type arrays, one-sided or nonnative integer bounds, nonstring enum values and tags, and noncanonical tuple/nullable/tagged forms remain refused as unsupported. Actual wrong keyword types remain malformed. Arbitrary third-party protocol schemas are outside the initial release contract.

BEHAVIOR

Profile restrictions do not claim malformed schema

Given A schema uses an otherwise valid form outside the finite profile.

When The application requests a runtime contract.

Then Loading refuses it as unsupported at the relevant path.

Then No broader schema form is admitted by changing the diagnostic.

level: boundary area: Document loading

BEHAVIOR

An unsupported keyword remains an error

Given A finite-profile schema includes an unhandled keyword.

When The application loads it as a contract.

Then Loading returns UnsupportedDocument at that keyword.

Then No contract with the constraint discarded is returned.

level: boundary area: Document loading

- Sources: loader, document cases, and contract cases.

1.7.3 Normalization and schema matching

- from_schema is total because every public Schema is checked. from_codec can fail with UnknownSchema and can panic on a source definition mistake. No process-local schema identity token is required.
- Normalization recursively sorts object properties by name, enum labels by text, and variants by tag. It retains associations and child order. Descriptions remain available for rendering; matching separately removes them recursively.
- Contract match is exact equality of those normalized shapes. It is not a solver for equivalence, subtyping, satisfiability, or data migration. A rebuilt equal contract can match a witness.

BEHAVIOR

Different bounds cannot reuse a witness

Given A value was validated under integer bounds zero through ten.

When The application decodes the witness with a codec declaring zero through eleven.

Then Decoding returns ContractMismatch before native conversion.

level: boundary area: Contract matching

- Bound equality must hold even when this particular value lies within both ranges. Matching a custom codec's declared schema still does not certify its executable callbacks. ADR-0006 records the witness contract.

1.7.4 Validation and native conversion

- Validation walks the value and returns the first located failure. It checks exact numeric integrality and bounds without native Int/Float projection; it checks exact pair length, list elements, enum labels, closed property requiredness, and tagged payload shape.
- Closed object and union validation reject duplicate or unknown names before validating declared children. Required properties and payloads retain Field paths; collections retain Index paths. AnySchema accepts the supplied value unchanged, including a manually constructed raw Object with duplicate members.
- Successful validation creates the opaque witness. contract.decode obtains the retained codec's schema, compares normalized shapes, then invokes that codec's native decoder. Schema mismatch and unavailable schema both become ContractMismatch.
- A witness for the mathematical integer 1e30 can still fail codec.int native decoding due to its 24-digit projection bound or JavaScript safety. A witness for a finite exact number beyond binary64 can fail codec.float with FloatOutOfRange. Those outcomes do not invalidate the mathematical schema.
- contract.value_codec supplies Codec(Value) with the contract's schema. Its decoder validates, while its encoder returns Value unchanged; invalid encodings remain possible. This adapter's current asymmetry is explicit in the pending ruling and cannot establish the full bidirectional schema law.
- Direct validate/load have no independent depth, width, or time guard. Values supplied through strict parsing inherit those parse bounds; manually assembled values do not. Consumers choose bounded construction/admission where their threat model requires it.

1.8 Legacy compatibility

The frozen 1.x surface retains established one-way decoding, independent encoders, broader schema description, and recursive references.

Legacy boundary

responsibility

Keep published combinators and established data readable without inferring modern codec laws.

interface

json/blueprint Decoder and FieldDecoder, decode0..decode9, field/optional_field, list, tuples, map, union/enum helpers, self_decoder, reuse_decoder; json/blueprint/schema.

interactions

gleam/json parses after a byte precheck. Legacy schema rendering is independent from native decoder behavior.

invariants

Legacy wire forms remain available explicitly. Opaque Decoder retains its decoder callback, schema description, and definitions.

failure

json.DecodeError and dynamic decode errors retain legacy semantics; no strict finite-contract acceptance is inferred.

Legacy Decoder and FieldDecoder

value-object · immutable · JSON contracts · owned by Legacy compatibility

Immutable one-way native decoder with an independent legacy schema description.

ATTRIBUTES

derived **Callback and description** · Dynamic decoder × SchemaDefinition × definitions
Combinators assemble conversion and description. FieldDecoder additionally retains its property name. No modern schema-validation witness is inferred.

RELATIONSHIPS

1 : 0..n Reusable decoders retain named definitions for references; lazy self-decoding resolves through the caller's retained decoder function.

- Legacy unions use type/data; legacy enum encoders use an enum object. Optional fields treat absence and null alike. Nonempty record decoding ignores unknown fields; decode0 permits scalar/nonempty input.
- decode uses a 1 MiB precheck before gleam/json. decode_with_max_bytes changes that bound. Native parsing owns duplicates and precision; there is no depth/value/token bound on this path.
- self_decoder accepts a lazy decoder and describes a root reference. reuse_decoder hashes the rendered schema definition with SHA-1 to name a `$defs` entry and rewrites root self-references to that entry. Nested recursion needs this explicit rebasing; the hash is a deterministic local description key, not a security authority or schema-equivalence proof.
- The legacy schema description supports type combinations, enum/const, nullable/optional, numeric constraints, string lengths/patterns/formats, array and object constraint families, allOf/anyOf/oneOf/not, references, and boolean schemas. This is a rendering surface, not the modern finite validator or proof that each described constraint is enforced by its decoder.
- The legacy renderer declares Draft-07 while some descriptions use later forms such as prefixItems and `$defs`. Consumers must not infer dialect correctness or general validation parity from that label. Changing only the label cannot prove compatibility.
- Published combinator callers remain supported; direct transparent record construction and imports from the former public dynamic module do not. The operational 1.x migration guide identifies source and data consequences. ADR-0007 records the compatibility boundary.
- Sources: legacy decoder, legacy schema renderer, compiled legacy examples, and current migration tests.

1.9 Source generation

Development tools return reviewable source. Runtime document loading never constructs a new native type.

1.9.1 Typed-definition generator

Codegen definition boundary

responsibility

Lower a typed definition to a standalone Gleam module with matching runtime behavior.

interface

Dev-only json_blueprint_codegen: Definition, named_mapping, enum_variant, runtime, compile.

interactions

The runtime codec is checked before lowering. Generated source uses runtime internal/generated helpers under the versioned compatibility contract.

invariants

Names/references are explicit. Generated applications do not depend on the generator package.

failure

CompileError preserves invalid names/references/imports, invalid definitions, unavailable schemas, unsupported constructors, and native-number restrictions.

Definition(a) value-object immutable JSON contracts owned by Source generation

Immutable source-lowering definition whose native type belongs to the compiled caller.

ATTRIBUTES

derived **Runtime and source model** · Codec(a) × native type expression × lowering function
Definition constructors compose a runtime codec and source instructions. Source names remain explicit input where functions cannot be inspected.

RELATIONSHIPS

1 : 1 Definition retains exactly one runtime codec. Its properties and mappings may retain multiple child definitions.

Build value	Owned data	Invariant and failure
Properties(r, a)	Record builder, type expression, names, property lowering	Duplicate names fail definition checking; object closes the accumulated fields

Build value	Owned data	Invariant and failure
Mapping(a, b)	Two runtime functions and two source references	Source forms validate; callback/reference behavioral equality remains caller-owned
EnumVariant(a)	Wire label, native value, source constructor reference	Reference syntax checks before admission; enum definition checks label/value uniqueness
GeneratedModule	Relative path, content, optional codec fingerprint	No file is written automatically; compile and materialization return distinct result records

- Codegen definition carries a Codec(a), a native type expression, and lowering instructions. Properties combines typed required/optional fields for a closed object. Mapping joins runtime functions with explicit qualified references because Gleam cannot recover source names from closures.
- Supported definitions are string, int, number, bool, integer ranges, pair, list, nullable, finite string enum, required/optional properties, object, imap with named_mapping, and descriptions. A definition can exist at runtime while its native generated path is unsupported; compile refuses exact-number forms lacking supported native lowering with NativeNumberUnsupported.
- compile validates module path and accessor name, parses native type references, calls codec.check, lowers the definition, checks native support and import agreement, then returns a Generated module. The result contains relative .gleam path, content, and SHA-1 content fingerprint.
- Generated schema helpers trust generator-checked trees. Direct caller construction through internal tree helpers bypasses ordinary schema admission and owns the resulting invariants.
- Qualified callback/constructor names and type/import references are validated as source forms. Compilation checks referenced function types; same-typed callbacks paired with different source references remain a caller trust boundary. Parity tests must compare interpreted and generated behavior.
- A generated module exports Value encode/decode, strict text encode/decode, the explicitly distinct native JSON decoder, a schema accessor, and a codec accessor. Strict generated decoding uses the common Value parser; native decoding uses gleam/json after a 1 MiB byte check and does not inherit strict depth/number policies.
- Generation returns source only. The caller owns file writing, formatting, compiling, reviewing, committing generated output, and drift checks. Package internal/generated helper names/signatures consumed by checked-in output remain stable within a major version; internal marking alone does not eliminate that compatibility obligation.
- Sources: generator, runtime helper contract, compiled source fixtures, and build workflow. ADR-0008 records ownership.

1.9.2 Schema materialization

- The internal schema materializer exports functions reconstructing checked schema structure without constructing codec callbacks. It validates module/accessor names, rejects duplicate accessor names, sorts exports by accessor name, and escapes labels as data.
- Materialized schema functions may allocate a schema tree on each call. They generate neither native record types nor native codecs, and they establish no constant-evaluation or constant-time lookup guarantee.
- Current lowering emits supported tree constructors and preserves descriptions, child schemas, property requiredness, and variant payload presence. Unknown or unsupported structure returns a located MaterializationError rather than dropping constraints.
- Sources: materializer and schema materialization evidence.

1.9.3 Schema-driven generation obligations

- Runtime-schema generation remains intended: an admitted finite contract can drive development-time native type and codec source. The build compiler creates the native type; an application does not obtain Codec(Order) by parsing a schema at runtime.
- Stable selected module/type/field/constructor names, shared named definitions, nested objects, collections, numeric enums, aliases, and recursive generation need contracts beyond the typed-definition generator. Unsupported shapes must return located failures without losing constraints.
- CLI workflow, committed-output drift checks, formatter integration, and upgrade fixtures remain retained tooling obligations. The pending generation entry preserves these requirements.

1.10 Schema extension families

The finite delivered language coexists with retained broader capability scope. An unsupported document remains raw data until a supported interpretation exists.

Family	Retained obligation	Unresolved design detail
String constraints	Length, patterns, and explicit format semantics	Unicode length policy, regex dialect/limits, annotation versus assertion
Numeric constraints	Broader bounds, exclusivity, and multipleOf	Exact arithmetic, projection distinction, hostile-resource budgets
Collections and objects	Additional item/property constraints and broader object models	Uniqueness, contains, pattern properties, limits and error selection
Enums and alternatives	Numeric enums, aliases, and general alternatives	Canonical encoding, overlap, branch failures, equivalence limits
References and recursion	Recursive codec/schema families and named definitions	Resolution scope, cycles, finite traversal/expansion, recursive laws
Raw unmodeled schemas	Retain data for consumer policy without claiming validation	Supported interpretation and admission remain explicit
Evolution	Schema and wire migration fixtures	Compatibility direction and accepted transformations

- Each added family must define encoding/decoding agreement, schema semantics, rejection behavior, finite resource policy, and independent oracle fixtures. No constraint may be silently discarded to accept a remote schema.
- Provider and protocol consumers own admission of a supported schema. They may reject AnySchema, tuple forms, optionality, unions, numeric precision, or future OtherSchema independently of generic local validity. Successful local loading is not proof of remote acceptance.
- The original broader schema description is preserved by the legacy surface, but its existence is not modern codec/runtime validation support. The pending broader-family entry identifies the build obligation. ADR-0001 records the rationale and scope provenance.

1.11 Verification and extension boundaries

Verification distinguishes finite acceptance, native projection, legacy behavior, generated parity, and executable documentation.

1.11.1 Oracle and behavioral evidence

- The independent oracle compares emitted and normalized schemas, codec decoding, and runtime validation with Python jsonschema Draft202012Validator against a frozen manifest. Selected cases carry stable identifiers and expected schemas/instances; missing, extra, duplicate, or replaced cases fail closed.
- The current source manifest contains 85 finite cases in 16 families. That count identifies the stored corpus rather than a new execution claim. Duplicate-key evidence is checked before native export can erase it. Exact-number kernel tests and target float-bit tests remain separate from this Boolean finite corpus.
- Constructor law tests cover admitted native encode/decode and valid re-encoding where applicable. Custom mapping, asymmetric Value adapters, and native projection constraints require their own explicit law scope.
- Hostile tests exercise parser grammar, resource bounds, duplicate keys, malformed/unsupported schemas, and deterministic generated inputs. Wide collection traversal uses tail-recursive paths where the implementation supplies them; caller-constructed unbounded trees retain no guaranteed stack bound.
- Sources: independent checker, frozen manifest, runtime cases, and hostile cases. Oracle instructions state reproducible commands and evidence limits; ADR-0002 records historical source attribution.

1.11.2 Native and generated boundaries

- Erlang/OTP and Node targets exercise the same public contracts with explicit safe-integer differences. Number FFI owns host byte handling, integer admission, binary64 parts, exact decimal/binary comparison, and exact decimal expansion. JSON FFI owns bounded byte checks and copied strings; internal dynamic FFI preserves legacy/native decoder conversion.
- FFI cannot trust an external host caller's Gleam type annotation. JavaScript adversarial tests inject nonfinite, fractional, unsafe, and previously rounded Int values. Native function name/arity changes need explicit upgrade audits because foreign callers bypass Gleam compiler checks.
- Generated fixture tests format and compare regenerated source, compile retained modules, compare schema bytes and behavior, and run both targets. Named mapping references need parity evidence beyond source type checking. Benchmarks are measurement fixtures rather than semantic limits.
- Sources: Erlang number FFI, JavaScript number FFI, Erlang JSON FFI, and JavaScript JSON FFI.

1.11.3 Documentation and build contracts

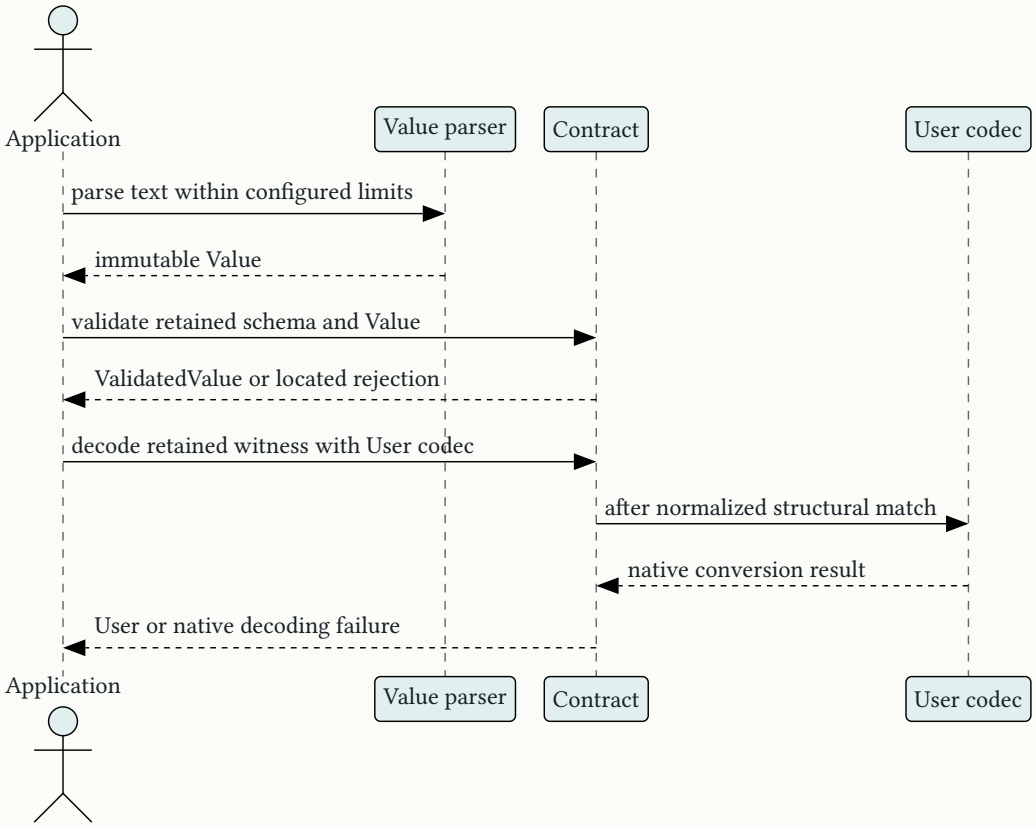
- README's ten Gleam snippets are compared verbatim against compiled and executed tests. Public module documentation examples and codegen README examples have the same executable drift checks. Legacy examples in docs/v1.md remain paired with test/examples.
- The package gate formats source/tests, builds with warnings as errors, and tests Erlang and JavaScript for root and codegen packages. It then runs the independent Python schema checker. Each package's own generated build artifacts are removed so obsolete compiled modules cannot hide missing source.
- Nix owns tool versions and the Python oracle environment. Design checks validate source shape, links, and rendered freshness; they do not establish general schema conformance or complete semantic fidelity. package gate and development environment define operational verification.
- Separate public consumers own ordinary tasks, advanced admission, caller types, and failure handling. Existing Relay/Fabric/LLM paths retain codecs for publication and conversion; their provider/protocol policies remain external. No shared execution or error runtime is introduced.

1.12 End-to-end walkthrough

A caller retains one User codec, admits text, publishes its schema, and can validate runtime data before native conversion.

- User has name:String, age:Int bounded zero through 150, optional email:String, and Role encoded as admin/member. The record builder ends in User(name:, age:, email:, role:) and supplies native getters for each field.
- The input {"name":"Ada","age":36,"role":"admin"} passes byte/depth/value/number admission, retains exact age, and decodes email as None. Encoding omits email. Schema publication includes closed properties, required name/age/role, exact age bounds, and the enum labels.

SEQUENCE Validation precedes native conversion



The sequence is synchronous. Callback behavior belongs to the retained native codec; no process or transport is implied.

- An age of 151 fails at Field("age") with IntegerOutsideRange. An explicit email:null fails unless that field's inner codec is nullable. A repeated age key fails text admission before conversion.
- A remote schema can be parsed only if it belongs to the finite profile. A default-open object is currently refused and has a pending classification ruling. A consumer needing broader remote schemas must preserve the unsupported result rather than substitute a permissive contract.

- Mapping User into an application-specific type retains the wire schema. The application handles Custom failures explicitly and owns mapping laws. Admitting a tool or provider request remains that consumer's separate decision.

Glossary

Every term the layer declares, with the context that owns it.

Codec

owned by root

A typed definition joining native encoding, native decoding, a placeholder, and an available or unavailable schema. The native type parameter belongs to the caller's compiled program.

Codegen definition

owned by root

A development-time value joining a runtime codec, a native type expression, and source-lowering instructions. Named callbacks include explicit source references.

Contract

owned by root

A normalized runtime-validation schema that retains no native type or application callback.

Contract match

owned by root

Equality of finite schema structure after normalization and removal of descriptions.

Definition error

owned by root

A contradiction in a codec definition, distinct from invalid input or unavailable schema.

Exact number

owned by root

A finite mathematical JSON number represented by a normalized decimal coefficient and exponent, with one positive zero. Its value is independent of a target's native integer or binary64 representation.

Finite profile

owned by root

The supported schema language of primitives, exact bounds, closed objects, pairs, lists, nullable values, string enums, tagged unions, and any value. It is narrower than JSON Schema Draft 2020-12.

Generated module

owned by root

A returned Gleam source artifact with its relative module path and, for codec generation, a content fingerprint. Writing and compiling it belong to the caller's build workflow.

JSON value

owned by root

An immutable JSON algebra value with ordered object members and an exact number alternative. Duplicate object members remain representable until a checked boundary rejects them.

Legacy decoder

owned by root

The frozen 1.x one-way decoder with an independent legacy schema description. Its established data envelopes and acceptance rules differ from Codec.

Native projection

owned by root

A checked conversion of an exact number into a target's Int or Float representation. Wire acceptance and successful projection are separate facts.

Placeholder

owned by root

A codec's native value used to describe a definition without input and to construct a failing native decoder. It need not be valid application data.

Record builder

owned by root

A continuation-based codec definition that binds decoded fields by name and ends with a native record constructor. Each field also supplies its native getter.

Schema

owned by root

An opaque, checked description of the finite JSON acceptance profile. Descriptions are annotations on that structure.

Schema view

owned by root

The stable public projection of one schema node with opaque schema children. Its fallback carries a future kind's rendered JSON Schema object.

Tagged union

owned by root

A closed object sum selected by a string tag, with a payload exactly when that selected variant declares one.

Validated value

owned by root

An opaque immutable pair of a JSON value and the normalized schema that accepted it. Its evidence applies to that value under that schema.